

¿Qué es un Autoencoder?
Un Autoencoder es una red neuronal que aprende una representación comprimida (codificación) de los datos de entrada. Está compuesto por:

- Encoder: Reduce la dimensionalidad del dato original.
- Latent Space: Representación comprimida.
- Decoder: Reconstruye el dato original desde la codificación.

¿Qué es un Perceptrón Multicapa?
Un Perceptrón Multicapa (MLP) es una red neuronal completamente conectada compuesta de:

- Capa de entrada: Recibe las características de los datos.
- Capas ocultas: Procesan los datos mediante pesos y funciones de activación no lineales.
- Capa de salida: Genera las predicciones finales.

Aspecto	MLP	Autoencoder	CNN
Tipo de datos	Datos tabulares o imágenes aplanadas	Imágenes aplanadas (reconstrucción)	Imágenes con estructura espacial intacta
Estructura principal	Capas densas conectadas	Encoder-Decoder	Capas convolucionales y densas
Propósito	Clasificación	Reducción y reconstrucción	Clasificación (visión computacional)
Capas comunes	Densas	Densas	Densas
Capas específicas	N/A	Latent Space	Conv2D, MaxPooling2D
Tamaño del modelo	Depende del número de capas/neuronas	Generalmente compacto	Más grande debido a convoluciones
Tiempo de entrenamiento	Rápido (menos capas y parámetros)	Moderado	Lento (más cálculos)

¿Qué parte de la arquitectura comparten los tres modelos?

Pista: Todos utilizan capas densas en algún momento, pero con propósitos diferentes. Los tres modelos comparten el uso de capas densas (fully connected layers) en su arquitectura.

Explicación:

En los Autoencoders, las capas densas se usan para comprimir y reconstruir los datos al pasar por el cuello de botella. En los MLPs (Multi-Layer Perceptrons), las capas densas son fundamentales para aprender patrones mediante la conexión de todas las neuronas entre capas. En las CNNs (Convolutional Neural Networks), las capas densas suelen estar al final para combinar las características extraídas por las capas convolucionales y hacer predicciones.

¿Qué modelo requiere más parámetros y por qué?

Pista: Considera el uso de filtros en las capas convolucionales y la cantidad de conexiones en capas densas. El modelo CNN generalmente requiere más parámetros.

Explicación:

Aunque las capas convolucionales reducen la cantidad de parámetros comparadas con las capas densas, cuando las CNNs incluyen muchas capas convolucionales con múltiples filtros grandes, el número total de parámetros puede superar al de los otros modelos.

Las capas densas de un MLP también tienen muchos parámetros porque cada neurona está conectada con todas las demás, pero no capturan características espaciales como las CNNs.

¿Qué modelo tiene más capacidad de capturar características espaciales y por qué?

Pista: Observa el uso de capas convolucionales en el CNN.

El modelo CNN tiene mayor capacidad para capturar características espaciales.

Explicación:

Las CNNs están diseñadas específicamente para procesar datos con estructuras espaciales, como imágenes o secuencias. Las capas convolucionales aplican filtros que identifican patrones locales (bordes, texturas, formas) y los combinan progresivamente para capturar relaciones espaciales más complejas. En comparación, los MLPs y los Autoencoders no tienen mecanismos específicos para analizar la disposición espacial de los datos. ¿Qué ventajas tiene un autoencoder frente al MLP en términos de procesamiento de datos no supervisados?

El Autoencoder tiene la ventaja de aprender representaciones útiles de los datos sin necesidad de etiquetas (aprendizaje no supervisado).

Explicación:

Un Autoencoder entrena comprimiendo los datos en un espacio de menor dimensión y luego reconstruyéndolos, lo que permite capturar características esenciales de los datos sin supervisión. El MLP, en cambio, requiere etiquetas para realizar tareas supervisadas, como clasificación o regresión, y no está diseñado para aprender representaciones intrínsecas de los datos por sí mismo.

Preguntas para reflexionar:
¿Cómo cambió el rendimiento después de agregar Dropout y regularización?

Antes de regularización y Dropout: El modelo puede tener un rendimiento muy alto en los datos de entrenamiento pero bajo en validación/prueba debido al sobreajuste.

Después: El rendimiento en el conjunto de validación/prueba mejora, pero podría haber una ligera disminución en el rendimiento del conjunto de entrenamiento debido a las restricciones adicionales.

¿Qué ocurre si el factor de regularización es demasiado grande?

Si el factor de regularización L2 es muy grande, el modelo penaliza excesivamente los pesos, lo que puede llevar a que todos los pesos sean muy pequeños.

Esto puede causar subajuste, donde el modelo no tiene suficiente capacidad para capturar patrones complejos, resultando en un rendimiento pobre tanto en entrenamiento como en validación/prueba.

Preguntas para reflexionar:
¿Cómo afecta el tamaño del espacio latente a la capacidad del autoencoder para reconstruir imágenes?

El tamaño del espacio latente afecta directamente la capacidad del autoencoder para reconstruir imágenes, porque define cuánto puede comprimirse la información y cuántos detalles pueden ser retenidos:

Espacio latente pequeño:

Se fuerza al autoencoder a comprimir la información en menos dimensiones, lo que puede resultar en una pérdida significativa de detalles. Esto puede llevar a reconstrucciones borrosas o imprecisas, especialmente para datos con alta complejidad (como imágenes de alta resolución o con muchas texturas). Sin embargo, un espacio latente pequeño puede ser beneficioso si el objetivo es extraer las características más importantes o eliminar ruido. Espacio latente grande:

Con un espacio latente más amplio, el autoencoder puede conservar más información y, por lo tanto, mejorar la calidad de las reconstrucciones. Esto es especialmente útil para datos complejos donde se necesita retener características detalladas. ¿Hay un punto donde un espacio latente más grande ya no mejora la calidad de las reconstrucciones?

Sí, hay un punto donde aumentar el tamaño del espacio latente ya no mejora la calidad de las reconstrucciones.

Razón:

Un espacio latente más grande puede llevar a que el autoencoder almacene información redundante o irrelevante, permitiendo incluso

que memorize los datos en lugar de aprender representaciones útiles. Esto puede:

Incrementar la complejidad del modelo sin una mejora significativa en la calidad de las reconstrucciones.

Causar sobreajuste en los datos de entrenamiento, afectando el desempeño en datos nuevos.

Indicador práctico:

Si las métricas de error de reconstrucción (como MSE o PSNR) dejan de mejorar significativamente al aumentar el tamaño del espacio latente, esto sugiere que se ha alcanzado el punto de saturación.

Preguntas para reflexionar:

¿Qué ventajas tiene usar un modelo preentrenado?

Menor tiempo de entrenamiento: El modelo ya aprendió características generales como bordes, texturas y formas a partir de grandes conjuntos de datos.

Mejor rendimiento en conjuntos pequeños: Se aprovechan características robustas y bien generalizadas, lo cual es útil si MNIST tiene un conjunto limitado.

Evita la necesidad de grandes recursos computacionales: No es necesario entrenar desde cero redes profundas, lo que ahorra tiempo y hardware.

¿Cómo afecta el congelar o descongelar capas del modelo base al rendimiento?

Congelar capas (Freezing):

Ventajas:

Reduce el tiempo de entrenamiento, ya que no se actualizan los pesos del modelo base.

Es útil si el conjunto objetivo (MNIST) es pequeño o similar al conjunto usado para preentrenar (ImageNet).

Desventajas:

Las capas base pueden no ajustarse completamente a las características específicas de MNIST, lo que limita el rendimiento.

Descongelar capas (Fine-tuning):

Ventajas:

Permite ajustar los pesos del modelo base para que aprendan características más específicas del conjunto objetivo.

Puede mejorar el rendimiento en conjuntos diferentes al original del modelo preentrenado.

Desventajas:

Incrementa el tiempo de entrenamiento y el riesgo de sobreajuste si el conjunto objetivo es pequeño.

Algoritmo Backpropagation:

1. Inicio
2. $n \leftarrow 1$
3. Inicializar θ random
4. Mientras (criterio de parada que no satisfaga)
5. Para cada muestra (x^*, y^*)
6. Ejecutar la red con X y obtener Y
7. Fin-Para
8. Actualizar los pesos con back-propagation
9. Fin-mientras
10. Fin

Pseudocódigo para una CNN

1. Inicializar pesos y biases para las capas convolucionales, de pooling y completamente conectadas.

2. Mientras no se alcance el número máximo de épocas o convergencia:

a. Para cada batch de datos de entrada:

i. Pasar los datos a través de:

- Capa Convolutiva: Aplicar filtros para extraer características locales.

- Función de Activación (ReLU).

- Capa de Pooling: Reducir dimensionalidad preservando características importantes.

ii. Aplanar la salida del bloque convolucional y pasarla a una o más capas completamente conectadas.

iii. Aplicar la función softmax (o similar) para obtener predicciones.

iv. Calcular el error con la función de pérdida (p. ej., cross-entropy para clasificación).

v. Retropropagar el error para actualizar los pesos usando optimizador (p. ej., SGD, Adam).

3. Retornar la red entrenada.

Pseudocódigo para un Autoencoder

1. Inicializar pesos y biases para las capas del codificador (encoder) y decodificador (decoder).

2. Mientras no se alcance el número máximo de épocas o convergencia:

a. Para cada batch de datos de entrada X :

i. Pasar X por las capas del codificador para obtener la representación comprimida Z :

$Z = \text{Encoder}(X)$

ii. Pasar Z por las capas del decodificador para reconstruir los datos:

$X' = \text{Decoder}(Z)$

iii. Calcular la función de pérdida (p. ej., error cuadrático medio entre X y X').

iv. Retropropagar el error para actualizar los pesos del codificador y decodificador.

3. Retornar la red entrenada, incluyendo las capas del codificador (para compresión) y decodificador (para reconstrucción).

Pseudocódigo para un MLP (Multilayer Perceptron)

Pseudocódigo para un MLP (Multilayer Perceptron)

1. Inicializar pesos y biases para las capas ocultas y la capa de salida.

2. Mientras no se alcance el número máximo de épocas o convergencia:

a. Para cada batch de datos de entrada (X, Y) :

i. Pasar X a través de:

- Capa Oculta 1: $Z_1 = \text{Activación}(W_1 * X + b_1)$.

- Capa Oculta 2: $Z_2 = \text{Activación}(W_2 * Z_1 + b_2)$.

- Repetir para más capas ocultas si las hay.

- Capa de Salida: $Y' = \text{Softmax}(W(X_{\text{final}}))$.

ii. Calcular el error con la función de pérdida (p. ej., cross-entropy para clasificación o MSE para regresión).

iii. Retropropagar el error para actualizar los pesos usando optimizador.

3. Retornar la red entrenada.

Preguntas de teoría [7 puntos]

1. **Explique en forma detallada cómo funciona el perceptrón.**

El perceptrón es un modelo de red neuronal que clasifica datos en dos clases. Recibe un conjunto de entradas, las combina con pesos asignados y les suma un valor adicional llamado sesgo. Luego, aplica una función de activación (como la función signo) para determinar si la salida pertenece a una clase o a la otra. Durante el aprendizaje, ajusta los pesos y el sesgo en función de los errores que comete, hasta clasificar correctamente los datos o alcanzar un límite de iteraciones.

2. **Defina con tus palabras en qué consiste el perceptrón multicapa.**

El perceptrón multicapa (MLP) es una red neuronal formada por múltiples capas de nodos: una capa de entrada, una o más capas ocultas y una capa de salida. Cada capa está completamente conectada a la siguiente, y utiliza funciones de activación no lineales (como sigmoide o ReLU). A diferencia del perceptrón simple, el MLP puede resolver problemas no linealmente separables gracias a las capas ocultas.

3. **¿Qué es la retropropagación y cómo funciona?**

La retropropagación es un algoritmo de aprendizaje utilizado para ajustar los pesos en redes neuronales multicapa. Su funcionamiento incluye:

○ **Propagación hacia adelante:** Se calcula la salida de la red a partir de las entradas y los pesos actuales.

○ **Cálculo del error:** Se calcula la diferencia entre la salida deseada y la salida obtenida.

○ **Propagación hacia atrás:** El error se distribuye hacia atrás a través de las capas ocultas, ajustando los pesos según el gradiente del error respecto a cada peso (descenso por gradiente). Este ajuste se realiza capa por capa.

4. **Explique con un ejemplo la diferencia entre un problema linealmente separable y no lineal.**

- Un problema **linealmente separable** ocurre cuando los datos de dos clases pueden separarse con una línea recta (o un hiperplano en dimensiones mayores). Ejemplo: clasificar puntos en un plano donde $(x>0, y>0)$ ($x > 0, y > 0$) son clase 1 y el resto clase 2.
 - Un problema **no lineal** ocurre cuando los datos no pueden separarse con una línea recta. Ejemplo: clasificar puntos dentro de un círculo como clase 1 y fuera del círculo como clase 2.
5. **¿Qué significa overfitting? Mencione algunos ejemplos.**
El **overfitting** ocurre cuando un modelo aprende demasiado bien los detalles y ruido del conjunto de entrenamiento, lo que disminuye su capacidad para generalizar a nuevos datos.
- Ejemplo 1: Un modelo de regresión polinómica que ajusta un polinomio de grado muy alto a datos ruidosos.
 - Ejemplo 2: Una red neuronal con demasiadas capas y nodos frente a pocos datos de entrenamiento.
6. **¿Qué es el criterio de convergencia del algoritmo de aprendizaje del perceptrón y qué garantiza?**
El criterio de convergencia del perceptrón establece que el algoritmo detiene el aprendizaje cuando clasifica correctamente todos los datos de entrenamiento o alcanza un número máximo de iteraciones.
- Garantiza que si los datos son linealmente separables, el perceptrón encontrará una solución que clasifica correctamente los datos.
 - Si los datos no son separables, el algoritmo no converge y sigue iterando.

2. Se necesita elaborar un modelo del perceptrón que clasifique los datos en 2 clases...

a) Algoritmo para el aprendizaje del perceptrón:

Inicializar pesos y sesgo aleatoriamente

REPEAT

Para cada dato (x, y_d) en el conjunto de entrenamiento:

Calcular $z = \sum (w_i \cdot x_i) + b$

Calcular salida: $y = \text{signo}(z)$

Actualizar pesos: $w_i = w_i + \eta \cdot (y_d - y) \cdot x_i$

Actualizar sesgo: $b = b + \eta \cdot (y_d - y)$

UNTIL no hay errores o iteraciones máximas

b) Condiciones para que el perceptrón separe datos en 2 clases:

- Los datos deben ser **linealmente separables**.
- Las clases deben estar bien definidas y sin solapamiento.
- La tasa de aprendizaje (η) debe ser pequeña para evitar oscilaciones.

3. Asumiendo que tenemos una red neuronal con una capa de entrada...

a) Algoritmo para el aprendizaje: clasificar los patrones de entrada en dos clases

Inicializar pesos aleatoriamente

REPEAT

Propagación hacia adelante:

Calcular salida de la primera capa usando activación signo

signo

Calcular salida de la segunda capa usando activación sigmoide

sigmoide

Calcular salida final usando tangente hiperbólica

Cálculo del error:

$\text{error} = y_d - y_{\text{pred}}$

Propagación hacia atrás:

Ajustar pesos de la capa de salida

Ajustar pesos de la capa oculta

UNTIL error total < tolerancia o iteraciones máximas

b) Condiciones y limitaciones:

- Condiciones:
 - Los datos deben ser representables mediante las funciones de activación utilizadas.
 - El tamaño del conjunto de datos debe ser suficiente para entrenar la red sin overfitting.
- Limitaciones:
 - La red puede ser incapaz de generalizar a datos fuera del dominio de entrenamiento.
 - Entrenamiento lento si hay demasiadas capas o nodos.

Preguntas de teoría [7 puntos]

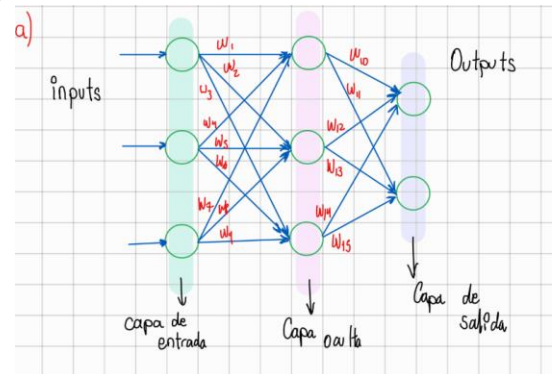
a) Describe brevemente qué es una MLP y cómo se diferencia de una red neuronal simple.

Una Red Neuronal Multicapa (MLP) es un modelo de red neuronal que tiene al menos una capa oculta entre la capa de entrada y la capa de salida. Se diferencia de una red neuronal simple (perceptrón) porque esta última carece de capas ocultas y solo puede resolver problemas linealmente separables.

b) ¿Cuál es la función principal de las capas ocultas en una MLP?

Las capas ocultas transforman las entradas en representaciones más complejas y no lineales, permitiendo que la red aprenda patrones difíciles de capturar con modelos simples.

a) Dibuja un esquema de una MLP con al menos una capa oculta.



b) Explica la función de cada capa en la arquitectura.

- Capa de entrada: Recibe los datos originales.
- Capas ocultas: Procesan los datos mediante combinaciones lineales y funciones de activación para extraer características relevantes.
- Capa de salida: Genera el resultado final, que puede ser una predicción o clasificación.

a) Nombra al menos tres funciones de activación comunes utilizadas en las capas ocultas de una MLP. Describe sus propiedades y cuándo se utilizan.

- Sigmoide: Produce valores entre 0 y 1, adecuada para problemas de clasificación binaria.
- ReLU (Rectified Linear Unit): Es eficiente y rápida, devuelve 0 para valores negativos y el valor original para positivos; muy usada en capas ocultas.
- Tangente hiperbólica (tanh): Genera valores entre -1 y 1, útil para modelar relaciones simétricas en datos.

a) Describe el proceso de propagación hacia adelante en una MLP. ¿Cómo se calcula la salida de la red a partir de las entradas?

La propagación hacia adelante consiste en calcular combinaciones lineales de las entradas con los pesos y sesgos, aplicando una función de activación en cada capa. Esto se repite capa por capa hasta obtener la salida final.

b) ¿Qué es la función de pérdida y cómo se relaciona con la salida de la red?

La función de pérdida mide la diferencia entre la salida de la red y los valores reales esperados. Es clave para evaluar el rendimiento del modelo y guiar el ajuste de los pesos.

a) Explica el algoritmo de backpropagation. ¿Cómo se calculan las derivadas parciales del error con respecto a los pesos?

Backpropagation calcula las derivadas parciales del error propagando hacia atrás el gradiente desde la salida hasta las capas anteriores, utilizando la regla de la cadena para actualizar los pesos.

b) ¿Qué es el descenso de gradiente y cómo se utiliza para actualizar los pesos de la red?

El descenso de gradiente es un método de optimización que ajusta los pesos de la red en dirección opuesta al gradiente del error, minimizando la función de pérdida iterativamente.

a) Menciona al menos dos técnicas de regularización utilizadas en MLPs. ¿Por qué son importantes?

Dropout: Desactiva aleatoriamente algunas neuronas durante el entrenamiento para prevenir sobreajuste.

L2 Regularización: Penaliza pesos grandes para evitar que el modelo se vuelva demasiado complejo.

b) ¿Cuál es la diferencia entre el descenso de gradiente estocástico (SGD) y el descenso de gradiente con momento (Momentum)?

SGD: Actualiza los pesos usando un subconjunto aleatorio de datos en cada iteración.

Momentum: Introduce un término que acumula la velocidad del gradiente para evitar oscilaciones y acelerar la convergencia.

a) ¿Cómo se inicializan los pesos de una MLP antes de comenzar el entrenamiento?

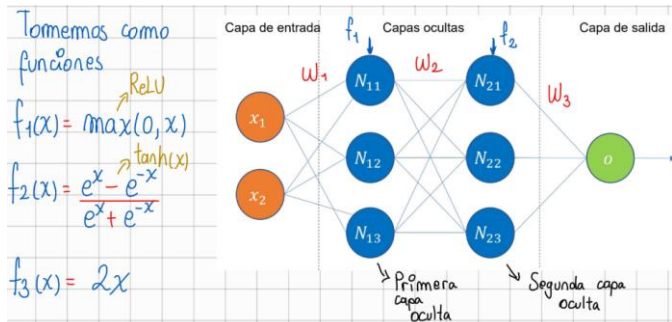
Se inicializan aleatoriamente o mediante métodos como Xavier o He, para garantizar una distribución equilibrada de los valores iniciales.

b) ¿Qué es el overfitting y cómo se puede evitar?

El overfitting ocurre cuando la red se ajusta demasiado a los datos de entrenamiento, perdiendo capacidad de generalización. Se puede evitar usando regularización, conjuntos de datos más grandes o validación cruzada.

1. Cálculo de la Salida de la Red (Propagación hacia Adelante)

- Dada las entradas ($x = [x_1, x_2]$), los pesos de la primera capa oculta ($W^{(1)}$) y las funciones de activación (f_1), calcula la salida de la primera capa oculta. [1.5 puntos]
- Si la segunda capa oculta tiene pesos ($W^{(2)}$) y función de activación (f_2), calcula la salida de la segunda capa oculta. [1.5 puntos]
- Finalmente, si la neurona de salida tiene pesos ($W^{(3)}$) y función de activación lineal, calcula la predicción final. [1.5 puntos]



$$\begin{aligned} a) \quad N_{11} &= f_1(w_{11}x_1 + w_{12}x_2) \\ N_{12} &= f_1(w_{13}x_1 + w_{14}x_2) \\ N_{13} &= f_1(w_{15}x_1 + w_{16}x_2) \\ b) \quad N_{21} &= f_2(w_{21}N_{11} + w_{22}N_{12} + w_{23}N_{13}) \\ N_{22} &= f_2(w_{24}N_{11} + w_{25}N_{12} + w_{26}N_{13}) \\ N_{23} &= f_2(w_{27}N_{11} + w_{28}N_{12} + w_{29}N_{13}) \\ c) \quad O &= w_{31}N_{21} + w_{32}N_{22} + w_{33}N_{23} \end{aligned}$$

2. Cálculo de Gradientes (Propagación hacia Atrás)

- Supongamos que el error en la predicción es (δ_{salida}). Calcula el gradiente de la función de pérdida con respecto a los pesos ($W^{(3)}$). [1.5 puntos]
 - Calcula el gradiente de la función de pérdida con respecto a los pesos ($W^{(2)}$). [1.5 puntos]
- a)** Sabemos que la función de activación $f_3(x) = 2x$ error en la predicción es δ_{salida}

Sea L la función de pérdida, entonces

$$\frac{\partial L}{\partial N_{salida}} = \delta_{salida}$$

gradiente respecto a los pesos $W^{(3)}$

$$\frac{\partial L}{\partial W^{(3)}} = \delta_{salida} \cdot a_2 \rightarrow \text{donde } a_2 \text{ es el vector de activaciones de la 2da capa oculta}$$

b) necesitamos considerar la propagación del error hacia atrás desde la capa de salida hasta la segunda capa oculta

i) error en la segunda capa oculta

$$\delta_2 = (\delta_{salida} W^{(3)}) \cdot f_2'(z_2)$$

donde $W^{(3)}$: son los pesos de la capa de salida
 $f_2'(z_2)$: es la derivada de la función de activación tangente hiperbólica aplicada a z_2

$$f_2'(z) = 1 - \tanh^2(z)$$

$$z_2 = \begin{pmatrix} w_{21}N_{11} + w_{22}N_{12} + w_{23}N_{13} \\ w_{24}N_{11} + w_{25}N_{12} + w_{26}N_{13} \\ w_{27}N_{11} + w_{28}N_{12} + w_{29}N_{13} \end{pmatrix}$$

ii) Gradiente respecto a los pesos $W^{(2)}$

$$\frac{\partial L}{\partial W^{(2)}} = \delta_2 \cdot a_1$$

a_1 es el vector de activaciones de la primera capa oculta.

3. Actualización de Pesos (Descenso de Gradiente)

- Si utilizamos una tasa de aprendizaje (α), ¿cómo se actualizan los pesos ($W^{(3)}$)? [1.5 puntos]
- ¿Cómo se actualizan los pesos ($W^{(2)}$)? [1 punto]

a) Se realizaría de la siguiente manera

$$W^{(3)} \leftarrow W^{(3)} - \alpha \frac{\partial L}{\partial W^{(3)}}$$

en donde α es la tasa de aprendizaje
y $\frac{\partial L}{\partial W^{(3)}}$ es el gradiente de la función de pérdida con respecto a los pesos $W^{(3)}$

b) Se realiza de la misma manera

$$W^{(2)} \leftarrow W^{(2)} - \alpha \frac{\partial L}{\partial W^{(2)}}$$

donde $\frac{\partial L}{\partial W^{(2)}}$ es el gradiente de la función de pérdida respecto a $W^{(2)}$

4. Regularización

- Supongamos que estamos utilizando la técnica de regularización L2 (Ridge) en una MLP. Si la función de pérdida (L) es la suma de errores cuadráticos (MSE) y los pesos ($W^{(2)}$) de la segunda capa oculta están siendo regularizados, ¿cómo se actualizan los gradientes durante la retropropagación? [1 punto]

a) La técnica de regularización añade un término de penalización proporcional a la suma de los cuadrados de los pesos. Definimos L_{reg} como la función de pérdida regularizada

$$L_{reg} = L + \frac{\lambda}{2} \sum_{i,j} (W_{ij}^{(2)})^2, \quad \lambda \text{ es el coeficiente de regularización}$$

y para actualizar los gradientes, tendremos al gradiente regularizado respecto a $W^{(2)}$

$$\frac{\partial L_{reg}}{\partial W^{(2)}} = \frac{\partial L}{\partial W^{(2)}} + \lambda W^{(2)}$$

∴ la actualización de los pesos $W^{(2)}$ será:

$$W^{(2)} \leftarrow W^{(2)} - \alpha \left(\frac{\partial L}{\partial W^{(2)}} + \lambda W^{(2)} \right)$$